Functorial Type Theory: An Executable Architecture for Directed Dependency

The emdash contributors

overview research article

> **Research-draft status.** This article describes the checked emdash v3.2 development and a bounded TypeScript reviewer. It is not a released foundation, a venue submission, or a claim that the full research programme is complete. The active Lambdapi sources remain the mathematical authority.

Abstract

Dependent type theory makes substitution computational. Category theory makes variation along arrows explicit. **Functorial Type Theory** asks what happens when both principles inhabit one language: variables may range over objects and arrows, dependent families carry transport, and functoriality and naturality are internal operations that can normalize rather than external proof obligations.

Emdash is an executable investigation of this idea. Its active Lambdapi development contains an autonomous categorical or directed dependent type theory of categories, Cat-valued families, reindexing, total categories, section categories, dependent homs, functors, and transfors. Lambdapi's dependent $\lambda\Pi$ framework supplies a complementary outer logical layer. A small TypeScript implementation mirrors this separation: an explicit locally nameless dependent Core checks and evaluates terms, while a typed categorical frontend compiles usable binder notation to existing internal categorical owners.

Two computations summarize the architecture. First, for $p : x \to y$ and $q : y \to z$, emdash forms the outgoing-arrow category

$$\mathrm{PathOut}_Z(x) = \Sigma_{y:Z}\, \mathrm{Hom}_Z(x, y)$$

and a canonical arrow $\rho_{x,y,p} : (x, \mathrm{id}_x) \to (y, p)$. Transporting a motive along ρ gives a synthetic arrow-induction principle. Applied to the representable composition motive, its checked normal form is $q \circ p$. Second, the TypeScript frontend accepts representative ordinary, natural, displayed-functorial, and displayed-natural abstractions. It recursively factors variable occurrences through weakening, pairing, evaluation, reindexing, totalization, and internal action owners, then emits backend-neutral explicit Core. The same checker accepts the result at object and arrow level; unsupported factorizations fail closed.

The result is a working research artifact rather than a completed proof assistant. It demonstrates a coherent design across mathematical kernel, dependent logical framework, elaboration, checked computation, and a client-side reviewer. Arbitrary displayed depth, whole-library transfer, complete groupoidal closure, and global metatheory remain explicit research boundaries.

1. Why Categorical Variables Should Compute

In ordinary dependent type theory, a bound variable may occur anywhere in a term, and the elaborator reconstructs applications and substitutions from local typing information. In ordinary category-theoretic prose, the same convenience is assumed for a richer notion of variable. A symbol may denote an object, an arrow, a functor, or a natural family; an expression such

as $F(x)$ silently invokes the appropriate object or arrow action. Naturality is usually discharged by saying that a construction is "functorial in x."

That phrase conceals an implementation choice. One can represent a construction as a pointwise function plus externally supplied equations, or one can represent it by an internal functorial object whose action and higher action are already part of the term. Emdash chooses the latter whenever a stable categorical construction is available. The aim is not to attach a naturality proof to every pointwise definition. It is to elaborate readable syntax into compositions of internal owners for which naturality follows from the type and generic action calculus.

This makes normalization do mathematical work. A composite of represented-hom actions is a cut; a rewrite that contracts it is cut elimination. A transformation component is not merely a projection from an opaque proof of naturality; it is an observable rung of an iterable hom tower. A dependent total arrow is not only a pair of endpoints; it contains a base arrow and a fibre arrow above transport.

The project therefore uses *dependent type theory* in two complementary senses.

1. An **outer dependent logical framework** supplies Π-types, λ-abstraction, application, definitions, and conversion. Lambdapi provides this layer for the authoritative development. The TypeScript Core implements a bounded counterpart with dependent checking and β/δ computation.
2. An **inner categorical or directed dependent type theory** treats a category as a context or shape and a Cat-valued functor as a dependent category over it. Reindexing is substitution, a total category is dependent sum, a section category is dependent product, and dependent hom records transport over a directed base arrow.

Neither layer replaces the other. The outer framework hosts the inner calculus; the inner calculus gives computational meaning to variables whose variation is genuinely categorical. The current groupoidal/type-theoretic universe supplies equality, J, dependent pairs, and dependent products as well, but systematically relating every directed construction to its groupoidal specialization is a later programme.

The central claim of this overview is deliberately narrower than a foundational completeness theorem:

> A substantial directed dependent calculus, a minimal outer dependent framework, and a recursive categorical-binder frontend already compose into one executable architecture.

The rest of the paper makes that claim concrete. Sections 2 and 3 describe the two layers and the directed dependent constructors. Sections 4 and 5 derive synthetic arrow induction and its composition normal form. Section 6 explains how ordinary and displayed categorical binders compile. Section 7 follows the result through explicit Core to the browser artifact. Sections 8 and 9 state the broader research programme and its present limits.

2. A Two-Layer Executable Architecture

The authoritative implementation is a Lambdapi signature. It declares the categorical owners and orients selected structural equations as rewrite rules. Proof-time unification rules relate useful presentations when neither should become the runtime normal form. Executable assertions in a separate checks module exercise formation, reduction, comparison, and non-collapse cases.

The TypeScript implementation is not a second mathematical authority. It is a small product kernel and elaboration target whose vocabulary is aligned with reviewed Lambdapi owners. Its trusted representation is explicit Core: applications have visible spines, binders are locally nameless, plicity and variation metadata are retained, and terms can be traversed, frozen, serialized, and compared structurally. A one-shot scoped builder restores HOAS-like authoring ergonomics without storing JavaScript closures in Core.

The architecture is:

```
 text or typed TypeScript surface
        ↓
scope, expected types, and contextual
occurrence analysis
        ↓
typed categorical wiring / abstraction
        ↓
backend-neutral explicit emdash Core
        ↓
small TypeScript dependent-LF checker and
evaluator
        └──────── optional deterministic
Lambdapi conformance oracle

active Lambdapi v3.2 source = mathematical
authority
```

2.1 The outer dependent LF

The outer Core has only the generic syntax needed to host object theories: sorts, free and bound variables, dependent products, lambdas, applications, metavariables, and owner calls. A dependent product stores the domain and a body under one binder. A lambda stores the same binder information and an explicit body. Bound occurrences use indices, so alpha-equivalence is structural and substitution is a total traversal rather than an invocation of an opaque host-language function.

Inference and checking are bidirectional. A free owner is looked up in the candidate environment; a call consumes its dependent product spine, instantiating each codomain with the checked argument. A lambda is normally checked against an expected product. An annotated lambda can also infer a product, which is useful for direct construction and diagnostics. Implicit parameters remain visible in Core even when a surface builder inserts them.

Conversion is deliberately bounded and evidence-producing. Weak-head β instantiates a lambda body and preserves any remaining application spine. Authorized transparent declarations unfold by δ only after their bodies have been checked against their declared types and their dependency order has been validated. Metavariables are zonked before rigid comparison, and Miller-pattern constraints are revisited as information arrives. One shared budget prevents a conversion query from hiding an unbounded reduction path; traces record whether progress came from β, δ, metavariable resolution, or a reviewed categorical runtime rule.

The outer TypeScript LF includes dependent Π, annotated λ-terms, application, contextual metavariables, transparent definitions, and bounded β/δ conversion. It deliberately rejects `Type : Type`; object theories use decoded codes in the same style as the Lambdapi development. The categorical layer does not add a new checker for each owner. Reviewed declarations and runtime or proof-time rules are compiled into the generic environment, after which ordinary LF inference, checking, conversion, and rewriting process the term.

This is enough to call the layer a minimal dependent logical framework, but not enough to claim a new general-purpose foundation. It does not implement an unrestricted user rewrite language, assume $\mathrm{Type} : \mathrm{Type}$, or silently treat every declaration as transparent. Those exclusions are part of the trusted boundary. The original TypeScript prototype had more globally mutable conveniences, including holes, rewriting, and HOAS storage; the renewed Core keeps the reusable algorithms while replacing global authority and opaque binder bodies with explicit, profile-scoped data.

The scoped builder reconciles the explicit representation with an ergonomic API:

```
 pi("x", A, x => B(x))
lam("x", A, x => body(x))
let_("x", value, x => body(x))
```

Each callback runs once. Its token is valid only during lowering and becomes a bound index immediately. A foreign or escaped token is rejected. Thus the authoring surface can look higher-order while the term that enters checking is first-order, deterministic, and serializable.

2.2 Semantic owners and transfer

The inner theory enters TypeScript through a typed immutable transfer representation. A declaration records its qualified owner, binder telescope, result, visibility, optional body, and source evidence. Runtime rules and proof-time comparisons are separate data classes. A policy overlay decides which reviewed items are executable in a candidate profile; it is not inferred merely because a Lambdapi symbol can be named.

That distinction mirrors the mathematical source. An opaque declaration extends typing but cannot unfold. A transparent definition participates in δ conversion. A runtime rule selects an operational normal form. A proof-time rule assists a rigid comparison without orienting runtime evaluation. Compiled module interfaces preserve public, protected, and private visibility rather than flattening every source declaration into one global namespace.

The initial directed continuation contains a small dependency-closed Sigma/Pi telescope rather than the entire library. Later qualification tranches exercised additional mechanisms—grouped rules, proof comparisons, source-ordered modules, generated inductive owners, internal Pi, Sigma-transfor operations, and profunctor fragments—through the same engines. This is the relevant sense in which transfer becomes mechanical: after mathematical review, adding an owner is a typed data and policy operation, not a new checker algorithm. It does not mean that an unreviewed rewrite rule acquires authority by being parseable.

This separation matters for scale. A categorical operation is represented by a semantic owner plus an argument schema, not a bespoke TypeScript AST tag with a private evaluator branch. Transfer infrastructure has already handled representative opaque and transparent declarations, grouped runtime rules, proof-time comparisons, source-ordered modules, and generated inductive owners. That evidence does not prove that every remaining Lambdapi declaration can be imported as one batch, but it establishes the intended unit of future work: reviewed data and policy added to generic engines.

The public text adapter is intentionally later and thinner. It recognizes a bounded mathematical syntax and resolves it into the same typed categorical surface used by direct TypeScript construction. Parsing a string does not create semantics. Expected types, classifier information, and the available internal owners decide whether an application denotes an object action, an arrow action, a displayed section component, a whole hom action, or a supported higher component. The adapter fails with a source location when no internal factorization exists.

This is also why Lambdapi remains useful after a browser-capable TypeScript checker exists. The TypeScript runtime makes the demonstrated profile small, inspectable, and client-side. Deterministic Lambdapi emission compares selected judgments against the active source during development. The oracle is a conformance route, not a production backend and not a substitute for the TypeScript checker.

3. Directed Families, Totals, And Sections

Let K be a category. A directed family over K is a functor $E : K \vdash \mathbf{Cat}$. It supplies a fibre category $E[k]$ for every object k, but also transport along every base arrow:

$$p : k \to k' \qquad \Longrightarrow \qquad E[p] : E[k] \vdash E[k'].$$

The arrow action is essential. A pointwise assignment of categories is not yet a directed dependent type. In the kernel, the stable category of such families is `Catd_cat K`; morphisms and higher morphisms are exposed by `Functord_cat` and `Transfd_cat`. Their ordinary Cat-valued presentations are available through controlled projections and proof-time comparisons, but the stable displayed heads are retained so that higher action remains iterable.

Reindexing along $F : A \vdash K$ is substitution:

$$F^{*}E[a] = E[F[a]],$$

implemented by `Pullback_catd`. The total category is the directed dependent sum:

$$\Sigma_K E.$$

Its objects are pairs (k, u) with $u \in E[k]$. An arrow $(k, u) \to (k', u')$ consists of

$$p : k \to k', \qquad \alpha : E[p](u) \to u'.$$

Thus the hom of a total category is organized by a dependent hom, not by an ordinary product detached from transport. The owner `sigma_arrow` forms (p, α), and `sigma_transport_arrow(E,p,u)` uses the identity fibre component to produce

$$(k, u) \longrightarrow (k', E[p](u)).$$

The dependent product is the category of sections:

$$\Pi_K E.$$

A section s assigns $s[k] \in E[k]$ and carries an action

$$s[p] : E[p](s[k]) \to s[k'].$$

The stable `Pi_cat` facade exposes section objects and the next displayed hom. Evaluation and section action project through the generic displayed application tower. They are not a second primitive function calculus.

3.1 Dependent hom as shared infrastructure

The common shape behind total arrows and section action is the dependent hom. Given

$$u \in E[x], \qquad v \in E[y], \qquad p : x \to y,$$

the fibre component of a directed arrow is

$$\mathrm{Hom}_{E[y]}(E[p](u), v).$$

As p varies this expression must itself carry action. Emdash packages it as a Cat-valued construction over the base hom, with variance chosen so that higher pre- and postcomposition normalize in the intended direction. The public `homd_` and internalized `homd_int` owners are therefore more than notation for a pointwise hom formula. They retain the object action, base arrow action, and the next hom needed by later consumers.

This internalization avoids a common design trap. Suppose one defined a displayed functor only by maps on fibre objects and fibre arrows and then attached an external naturality square. Such a package would be awkward to iterate: the next categorical dimension would have to unpack the square and rebuild its action. In emdash, the relevant object is already a `Functord_cat` or `Transfd_cat` term. Its action is observed through generic `fapp*`, `tapp*`, and displayed internal-hom projections. Higher consumers receive a first-class categorical term rather than a pointwise function plus evidence.

The same principle explains several otherwise technical owners. `sigma_map_func` turns a displayed functor into a functor between total categories, with its fibre action ending in the displayed internal-hom projection ladder. `sigma_pullback_total_func` maps the total of a reindexed family back to the original total:

$$\Sigma_A(F^* D) \longrightarrow \Sigma_K D, \qquad (a, u) \longmapsto (F[a], u).$$

Its arrow action sends (p, α) to $(F[p], \alpha)$. The construction is asymmetric because one input is a family reindexed along a specified functor; it is not a generic pullback of arbitrary total functors.

Likewise, section evaluation at an object and section action along an arrow are related observations of one coherent section. The fixed-index `piapp0` view is useful at the surface, while the generic displayed application tower retains its arrow and higher action. This is why the directed calculus deserves to be read as a DTT in its own terms: substitution, context extension, dependent programs, and their directed action are part of one computational structure.

These operations iterate to genuine dependent telescopes. If $R : K \vdash \mathbf{Cat}$, then $\Sigma_K R$ is the extended directed context. A second family depending on both k and $r \in R[k]$ becomes a family over this total category. The owners `Sigma_catd_functord_catd` and `Sigma_transfd_funcd` internalize the corresponding family and transformation uncurrying.

Further `Sigma_cat` and `Pi_cat` applications extend the telescope or form its dependent programs.

Independent displayed variables over one base use fibrewise products without postulating a new primitive product-family head. For $B, C : K \vdash \mathbf{Cat}$, write

$$P(B, C)[k] = B[k] \times C[k].$$

The active presentation is built from ordinary product formation, uncurrying, and the displayed family structure. Its projections, pairing, swap, and diagonal support weakening, exchange, and contraction among independent fibre siblings. This does **not** license swapping variables across a genuine dependency edge. In a telescope

```
 k : K;
a : A[k];
b : B[(k,a)], c : C[(k,a)];
d : D[((k,a),(b,c))]
```

b and c share a base and may be paired or exchanged; a cannot be moved past data whose type depends on it. The distinction is the familiar structural discipline of dependent type theory, now interpreted over objects and arrows of a category.

4. Synthetic Arrow Induction

The mathematical anchor of v3.2 is a directed analogue of path induction. Fix x in a category Z. The covariant representable family is

$$\mathrm{Rep}_Z(x)[y] = \mathrm{Hom}_Z(x, y).$$

Its total category is the category of outgoing arrows:

$$\mathrm{PathOut}_Z(x) = \Sigma_{y:Z}\, \mathrm{Hom}_Z(x, y).$$

An object is a pair (y, p) with $p : x \to y$. The distinguished object is the reflexive arrow

$$\iota_x = (x, \mathrm{id}_x).$$

Every outgoing arrow has a canonical total arrow from ι_x:

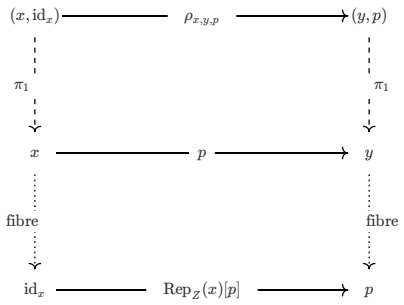
$$\rho_{x,y,p} : \iota_x \longrightarrow (y, p).$$

It is not postulated as a new path constructor. It is ordinary Sigma transport in the representable family:

$$\rho_{x,y,p} = \mathrm{sigmaTransport}(\mathrm{Rep}_Z(x), p, \mathrm{id}_x).$$

The endpoint is correct because representable action computes:

$$\mathrm{Rep}_Z(x)[p](\mathrm{id}_x) = p.$$



Now let $E : \mathrm{PathOut}_Z(x) \vdash \mathbf{Cat}$ be a motive and $u \in E[\iota_x]$. Transport along the canonical arrow defines a section:

$$\mathrm{Ind}_x(E, u)[(y, p)] = E[\rho_{x,y,p}](u).$$

In kernel notation, `path_ind_sec(Z,x,E,u)` inhabits

$$\Pi_{q:\mathrm{PathOut}_Z(x)} E[q].$$

This is fixed-source arrow induction. It resembles identity elimination because the reflexive outgoing arrow determines all components, but no invertibility is

assumed. The comparison from reflexivity to (y, p) is a directed arrow in a Sigma category.

The principal theorem internalizes the source x as well. Precomposition by $r : x \to y$ gives

$$\mathrm{PathOut}_Z(r) : \mathrm{PathOut}_Z(y) \vdash \mathrm{PathOut}_Z(x).$$

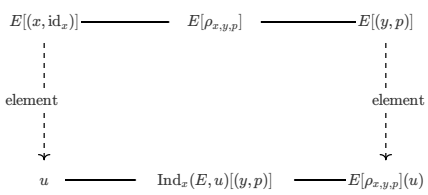
$$\mathrm{PathOut}_Z(r)[z, q] = (z, q \circ r).$$

Consequently motives vary by pullback and sections vary by section pullback. The source-indexed induction theorem is a displayed transformation

```
PathInd(Z) :
  PathOutReflEval_Z => PathOutPi_Z
```

Its kernel owner is `PathInd_transfd(Z)`. At each x it is the fixed-source induction functor. Along an arrow r, its type already contains the comparison between transporting initial data and pulling back the resulting section. Naturality is therefore internal to one displayed transformation; it is not an externally supplied square.

The fixed-source component can be pictured as motive transport:



There are two different kinds of non-strictness here. First, changing the source object acts contravariantly on `PathOut`, so the target section is pulled back rather than transported by an identity. Second, an arbitrary displayed functor need not preserve a transported Sigma arrow as a strict cartesian map; its component-level comparison remains visible through the displayed internal-hom action. The theorem packages the comparisons that are actually justified. It does not obtain a stronger strictness principle by calling the construction "induction."

There is also a totalized presentation over the Sigma category of sources and motives. It is derived by applying `Sigma_transfd_funcd` to the telescope theorem:

```
 PathInd_funcd(Z) =
Sigma_transfd_funcd(PathInd_transfd(Z)).
```

The direction of ownership is significant. The displayed transformation is primary because it retains the varying-source action. The total functor is a useful derived view, not a second induction axiom.

5. Composition As A Checked Normal Form

Arrow induction earns its keep by computing a familiar operation. For $p : x \to y$, define a motive at (y, p) by the functor category between representables:

$$\mathrm{CompMotive}_x[(y, p)] = \mathrm{Rep}_Z(y) \vdash \mathrm{Rep}_Z(x).$$

At the reflexive outgoing arrow this is $\mathrm{Rep}_Z(x) \vdash \mathrm{Rep}_Z(x)$, so the initial datum is the identity functor.

Induction produces a section

$$\mathrm{pathCompSec}(x) = \mathrm{Ind}_x(\mathrm{CompMotive}_x, \mathrm{id}).$$

Its component at p is a functor

$$\mathrm{pathComp}(p) : \mathrm{Rep}_Z(y) \vdash \mathrm{Rep}_Z(x).$$

Applying it to $q : y \to z$ normalizes to composition:

$$\mathrm{pathComp}(p)[z][q] \rightsquigarrow q \circ p.$$

5.1 The normalization route

The final equation is compact, but the checked route crosses several abstraction boundaries. At a high level, evaluation performs the following steps:

```
 PathInd_transfd(Z)
   ↓ component at x
PathInd_func(Z,x)
   ↓ component at CompMotive_x
path_ind_sec(Z,x,CompMotive_x,id)
   ↓ section component at (y,p)
CompMotive_x[rho_{x,y,p}](id)
   ↓ representable and Sigma-transport
computation
path_comp_func(p)
   ↓ action at (z,q)
hom_postcomp_fapp0(id_Z,q,p)
```

Each arrow is owned by a reusable projection or computation. The theorem's displayed component projects to fixed-source induction. Section evaluation projects to motive transport along ρ. Sigma transport computes the endpoint from $\mathrm{Rep}_Z(x)[p](\mathrm{id}_x) = p$. The composition motive then exposes postcomposition between representables.

This factorization is valuable for regression. A direct rule `pathComp(p,q) -> q o p` would prove only that one named head reduces. The expanded checks verify that the theorem remains connected to its semantic construction. They instantiate both the displayed telescope route and the derived total-functor route all the way to the same hom-action owner. Nearby negative assertions keep opaque motives, unrelated arrow endpoints, and higher cells from being consumed by the specialized path.

The checks exercise this result both through the primary telescope theorem and through the derived Sigma-total theorem. They also check the preceding representable and ρ computations, so the final result is not an unrelated rewrite attached directly to the theorem name.

There is a useful implementation subtlety. The runtime normal form is the represented-hom postcomposition owner

```
hom_postcomp_fapp0(id_Z, q, p).
```

The ordinary categorical presentation `comp_fapp0(q,p)` is a typed proof-time comparison surface. Emdash does not globally erase these provenances into one raw composition syntax. Postcomposition, precomposition, and rigid two-endpoint hom action have different variance and different higher behavior. Narrow comparisons relate them when a theorem needs to see the same categorical composite.

This illustrates the normalization policy used throughout the kernel.

- A **rewrite rule** selects a runtime normal form and participates in reduction and critical-pair analysis.
- A **unification rule** helps the framework compare two intended presentations when neither should compute to the other.
- The global `fapp*` and `tapp*` calculus owns generic identity, composition, functoriality, and naturality.
- Constructor-specific rules are projection betas or measured joins, not duplicate copies of generic coherence.

In this sense, the equation $\mathrm{pathComp}(p)(q) = q \circ p$ is both a mathematical theorem and a regression test for the architecture. It passes only if Sigma transport, representable action, displayed transformation projection, section evaluation, and hom-action cut elimination agree on a normal form.

6. Usable Categorical Binders

The kernel expressions above are explicit and compositional, but they are not how an end user wants to author every program. Ordinary dependent type theory allows a variable to occur recursively inside a term and reconstructs the wiring. Categorical syntax needs the same usability while distinguishing several kinds of action.

The current surface marks the intrinsic binder mode on the lambda:

```
 λ^f   functorial
λ^n   natural / indexed
λ^fd  displayed-functorial
λ^nd  displayed-natural
```

A type annotation such as `x : A` is separate and may be inferred when the expected type determines it. This is why `λ^f x. ...`, rather than `λ x :^f A. ...`, is the durable notation direction: functoriality belongs to the binder, while the domain annotation is ordinary bidirectional typing information.

6.1 Application is type-directed

Whitespace application is intentionally neutral in the source. The expression `F x` does not itself say whether to use an object component, a capped arrow component, a whole hom action, a section evaluation, or a displayed component. The resolver combines the inferred type of `F`, the classifier and variation of `x`, and the expected result.

Typed situation	Selected internal reading
`F : A ⊢ B`, `x : Obj(A)`	object action `fapp0(F,x)`
`F : A ⊢ B`, `p : Hom_A(x,y)`	arrow action `fapp1(F,p)`
functor expected over a whole hom	internal hom action, not one capped point
`s : Π(k :^n K), E[k]`, `k : Obj(K)`	section component `piapp0(s,k)`
`FF : E ⊢_K D`, `a : E[k]`	displayed fibre component
coherent transfors at `k`	internal component and higher-action owners

The whole-hom case is particularly important. If the expected result is a functor between hom-categories, prematurely choosing the value on one arrow would lose the object needed for the next higher action. The resolver therefore preserves a functor-level term whenever later iteration is requested. This is the surface counterpart of the kernel's preference for functor-level folds over capped point rules.

Contravariance is also typed rather than guessed from names. Represented hom has distinct postcomposition and precomposition owners. A future surface may infer polarity from a classifier or expected mixed-variance family, but the current frontend supports only reviewed cases. It will not reverse an arrow or synthesize an opposite merely because doing so would make a local application typecheck.

Neutral application makes the syntax familiar while keeping the elaboration falsifiable. Each selected route produces an explicit owner that the generic checker can verify. When two routes are possible at the raw syntactic level, the expected type disambiguates them; when the expected type is insufficient, the diagnostic requests an annotation rather than committing to a noncanonical action.

The frontend does not recognize whole source strings and replace them with owner-specific templates. It lowers each scoped callback or parsed binder to a first-order contextual representation and recursively analyzes subexpressions. Variable occurrence and expected classifier information determine structural wiring.

For ordinary functorial abstraction, the basis includes:

- identity when the body is the variable;

- weakening when the variable is unused;
- diagonal/contraction when it is used more than once;
- exchange for independent nested variables;
- product pairing and projections;
- evaluation of a functor-valued expression against an argument expression;
- functor composition; and
- curry/uncurry for nested abstraction.

This handles examples that are easy to write but not reducible to a single eta pattern:

```
λ^f x. (H x) (K x)
λ^f x. F x y0
λ^f x : A. λ^f y : B. E y x
```

The first uses x in both function and argument positions. The second abstracts after evaluation at a fixed inner object. The third exchanges independent variables before evaluation. Each compiles to existing product, pairing, evaluation, composition, and exchange owners.

6.2 Dependency-aware structural compilation

The compiler treats a context as ordered typed slots plus dependency edges. For each abstraction it first lowers the body recursively, then computes how the selected slot occurs in the resulting typed tree. Zero, one, or multiple uses select weakening, identity-like routing, or contraction/pairing. Independent nested slots may be exchanged. A slot that occurs in the type of a later slot cannot be exchanged across that dependency.

Displayed contexts add a base classifier to every slot. Extending by $a : A[k]$ changes the base from K to $\Sigma_K A$. A later family must name that total or a checked pullback of a family along a known projection. Independent siblings B and C over the same extended base are grouped through $P(B, C)$; after their pair is introduced, the next base is $\Sigma P(B, C)$. The compiler derives these transitions from family types, not from untyped punctuation alone.

This design supports both DTT viewpoints that arise in practice. A sequential telescope represents variables whose types depend on earlier variables. Fibrewise product structure represents several variables that are independent of one another but share a dependent base. Weakening, symmetry, and contraction apply inside the latter block, while substitution and totalization connect the former levels. The implementation may use different lowering routines for the two cases; what matters is that they compose through the same explicit internal owners without an ad hoc semantic exception.

Natural and displayed binders require indexed classifiers rather than an external naturality proof. A section component example is:

```
λ^n k : K. (FF k) (s k)
```

Here `s k` is a section value in a fibre and `FF k` is the component of a displayed functor. Recursive application elaborates the body through generic composition at `Catd_cat K`. Both the object component and the action over a base arrow are already owned by the internal displayed construction.

A displayed functorial example composes internal displayed functors:

```
λ^fd a : E. GG (FF a)
```

Displayed weakening can recover the hidden base index and apply a section:

```
λ^fd a : E. s (indexOf a)
```

For independent siblings over one base, the frontend uses fibrewise pairing:

```
λ^fd (b : B, c : C). fibrePair (FF b) (GG
c)
```

The mixed telescope combines genuine dependency edges with an independent middle block:

```
λ^fd (a : A; b : B, c : C; d : D).
fibrePair b c
```

Semicolons advance the dependent context. The comma says that b and c are siblings over the same preceding total. The family of d is based on the total containing their pair. The contextual compiler derives the necessary pullbacks and totalizations, while the user writes the variables in the same dependency order as an ordinary DTT telescope.

6.3 A worked mixed telescope

The mixed example is small enough to run in the reviewer but rich enough to show the dependency algorithm:

```
λ^fd (a : A; b : B, c : C; d : D).
fibrePair b c
```

Assume

```
K : Cat
A : Catd K
B, C : Catd (Sigma_cat A)
D : Catd (Sigma_cat P(B,C)).
```

The expected type says that the abstraction is a displayed contextual functor, not an outer LF lambda or a pointwise function. Lowering proceeds in four stages.

1. **Extend by the first dependent slot.** The compiler checks that A is a family over K and changes the active base to $\Sigma_K A$. The token a records both its fibre classifier and the projection back to K.
2. **Form the independent sibling block.** Both B and C are checked against the same active base. Their context is represented by the fibrewise product $P(B, C)$, with `Product_projL_funcd`, `Product_projR_funcd`, and `Product_pair_funcd` as structural owners. The base for the next level becomes its Sigma total.
3. **Check the final dependency.** The family D must be based on that exact total. A family merely over K, or over $\Sigma_K A$ before the sibling pair, is rejected. The body does not use d, so its route contains dependency-aware weakening rather than deleting the level from the telescope.
4. **Compile the body.** `fibrePair b c` uses the two sibling projections and the displayed pairing owner. The result family is the appropriate reindexing of $P(B, C)$ along the contextual projection. Core emission makes the pullbacks, functor compositions, Sigma projections, and product pair explicit.

The object component alone would not validate this construction. Let p be a base arrow and let u be an internalized arrow in the displayed source. The capped cell of the pairing owner reduces componentwise:

$$\mathrm{cell}(\langle FF, GG\rangle, p, u) = \big\langle \mathrm{cell}(FF, p, u), \mathrm{cell}(GG, p, u)\big\rangle.$$

The active kernel expresses this at the generic `fdapp1_int_cell` projection for `Product_pair_funcd`; it does not introduce a second product-cell calculus. The TypeScript consumer transfers the existing owner and rule, checks the object and internalized-arrow observations, and retains an opaque cell as a non-collapse witness.

Several errors are consequently structural rather than cosmetic. Giving B and C different bases invalidates the sibling block. Basing D on a total that omits the pair invalidates the dependency edge. Exchanging a past $B(a)$ is ill typed, while exchanging b and c is meaningful because they are independent over the same base. Returning a value from an unrelated displayed family fails expected-family checking even if its pointwise fibre happens to look similar.

This example also explains why "uniform" is not a requirement on the implementation technique. Sequential dependency and fibrewise sibling structure may use different compilation routines. The architectural requirement is that both routes compose naturally, scale beyond a hard-coded body shape, and end in internalized owners whose object and arrow actions are checked by the same Core.

The displayed-natural slice demonstrates recursive component composition:

```
λ^nd k : K. composeCells (theta k) (eta
k)
```

The result is not a pointwise function accompanied by a hand-authored naturality equation. The typed inputs are already transfors, their component operations are internal projections, and generic higher action carries the construction forward.

These examples expose the design's key usability rule:

> Convenient syntax may hide structural categorical operations, but it may not fabricate categorical coherence.

If the expected type requests an action for which the active kernel has no internal owner, elaboration reports an unsupported action. Escaped binder tokens, wrong families, incompatible bases, illegal dependency exchange, and foreign construction environments are rejected. This fail-closed policy is what allows the surface to grow without becoming a second, unsound category-theory implementation.

The string syntax is correspondingly bounded. The adapter parses neutral application, annotations, the four binder modes, and the reviewed constructors needed by ten examples. It does not parse arbitrary Lambdapi source or every notation in the book. Direct typed TypeScript remains the more complete construction surface; both routes converge before checking.

7. From Surface Terms To Explicit Core

Consider the ordinary source:

```
λ^f x. (H x) (K x)
```

The parser identifies a functorial binder and neutral applications. The resolver uses the expected functor type and the types of `H` and `K` to classify both occurrences of x. The contextual compiler constructs the diagonal, pairs the two resulting functorial expressions, and applies the evaluation functor. Serialization then reveals the explicit owners and arguments that were silent in the source.

The same path applies to direct TypeScript construction. A scoped builder invokes each callback once with an opaque token. Lowering replaces that token by a locally nameless contextual slot. No arbitrary JavaScript closure enters Core, and a token cannot escape its builder or be reused in another environment.

Explicit Core is then processed by one generic dependent-LF runtime:

1. declarations establish owner types and optional transparent bodies;
2. bidirectional checking verifies applications and dependent binders;
3. metavariable constraints are revisited through bounded pattern unification;
4. β reduces outer lambdas and δ unfolds authorized definitions;
5. reviewed categorical runtime rules normalize internal owners; and
6. the result is serialized with its inferred and expected type.

The strongest small dependent demonstration crosses both layers. An outer LF lambda receives a section over a nested Sigma telescope. Its body applies the section to a dependent pair. Checking verifies that the pair belongs to the expected family; evaluation performs outer β and then the directed Sigma-telescope fibre rule. Replacing the fibre object by one from another family produces a type mismatch rather than a stuck or coerced term.

7.1 Checked behavior, diagnostics, and provenance

Positive examples alone would not establish the boundary. The executable corpus pairs them with negative and non-collapse cases. An object from the wrong category cannot be passed to a functor action. A dependent pair from the wrong family cannot be

consumed by a section. A displayed family based on K cannot masquerade as one based on $\Sigma_K A$. An internalized arrow is checked not to collapse to its object component, and an opaque higher cell remains opaque when no projection rule applies.

Diagnostics are normalized at the surface boundary. They record a stable code, phase, source span, expected classifier or family, and an explanatory detail. The browser does not duplicate these checks; it formats the same diagnostic returned by the TypeScript elaborator. This makes an edited example useful to a reviewer: rejection exposes which architectural condition is missing instead of producing a generic JavaScript exception.

Conformance evidence is separate from runtime provenance. A candidate profile records the Lambdapi module, source or canonical-export evidence, selected owner set, and reviewed rule set from which it was built. Node-side tools can emit a deterministic Lambdapi judgment and compare it with the active kernel. The browser consumes only the frozen browser-safe selection contract and compiled runtime data; it neither reads source files nor computes an authority digest.

The browser reviewer packages this architecture without adding semantic machinery. Its expression panel offers ten checked presets across `^f`, `^n`, `^fd`, and `^nd`, editable text, explicit Core, inferred type, structural prerequisites, and source-located rejection. Its evidence panel runs the existing outer-LF, ordinary bracket, and genuinely displayed dependent witnesses on request. Its Core panel exposes a minimal editable dependent-LF script. The entire published runtime is client-side; it opens the overview paper and full book as static assets.

The artifact boundary is worth stating precisely.

Layer	What it establishes
Active Lambdapi sources	Authoritative categorical declarations, computation rules, and proof-time comparisons
Lambdapi checks	Executable positive, negative, and normal-form evidence for the active kernel
TypeScript Core	A small dependent checker/evaluator for the reviewed transferred profile
Categorical frontend	Recursive compilation of the demonstrated binders to internal owners
Browser reviewer	Reproducible access to the same TypeScript paths, with no server or production Lambdapi process

This is not a claim that the browser contains the entire Lambdapi development. It is a claim that the demonstrated end-to-end path is real: surface syntax reaches explicit internal structure, generic checking, and computation in the same client an external reviewer can run.

8. A Wider Computational Programme

Synthetic arrow induction and binder elaboration are representative computations, not the full scope of the kernel. The active categorical development also contains a broader calculus built with the same ownership discipline.

Cat-valued profunctors are represented as directed families over $A^{\mathrm{op}} \times B$. Representables act by the internal two-sided hom action. Endpoint reindexing, fixed-endpoint tensor, co-Yoneda maps, and covariant and contravariant implication expose selected computational interfaces. Weighted limits are

formulated as computational comparisons between a weighted-cone profunctor and a representable. Their push and pull maps cancel on arbitrary probes; right-adjoint preservation is assembled from adjunction mates and comparison composition. Weighted colimits and left-adjoint preservation are obtained by opposite normalization rather than a duplicate cut calculus.

The strict comparison object `DefIso` is characteristic of this style. It packages forward and backward categorical maps whose selected cuts compute. Public profunctor comparisons reuse it, so theorem-specific push/pull rules do not proliferate. A primitive directed join category supplies a further stress test: its cross arrows are one internally natural profunctor cell, and its recursor computes on both inclusions and the cross cell.

At the groupoidal end, the development includes decoded type codes, equality and J, dependent pairs and products, path actions, truncation levels, and staged equivalence/univalence interfaces. A walking-endomorphism development adds a concrete directed normalization cell before extracting equality by hom-discreteness. These results make the project more than a collection of frontend examples, while also showing why the overview should not pretend to finish the theory: several bridges among groupoidal, categorical, and weak higher-categorical structure remain research problems.

The architectural conjecture is that one internal language can support these constructions without a separate coherence mechanism for each feature. Generic action owns functoriality and naturality; semantic constructors own their projections and computation; elaboration reconstructs structural wiring; and normalization exposes observable results. The present artifact supplies nontrivial evidence for that conjecture, not its final proof.

9. Research Boundaries

The word *checked* in this article has a local meaning. Lambdapi accepts the active declarations and rules and verifies the recorded assertions; the TypeScript runtime accepts the demonstrated transferred terms and rejects its negative corpus. It does not mean that a global metatheory has already been formalized.

The principal open boundaries are:

- **Surface generality.** The categorical text language covers the reviewed examples, not arbitrary book notation, Lambdapi syntax, or every direct TypeScript constructor.
- **Displayed depth and variance.** Independent siblings, genuine dependency, a mixed `a; b,c; d` telescope, and selected higher action are implemented. Arbitrary telescope depth, all dependency-aware structural operations, polarity-directed contravariant lowering, and general displayed-transfor coherence are not.
- **Transfer scale.** Generic transfer engines and varied representative tranches exist, but systematic transfer of the whole active Lambdapi library has not graduated. Mathematical rules will continue to require owner and interaction review even if their representation becomes mechanical.
- **Categorical closure.** General dependent adjunctions $\Sigma_F \dashv F^* \dashv \Pi_F$, full profunctor tensor/coend semantics, complete equipment coherence, and semantic collage or dependent elimination for join remain future work.
- **Groupoidal closure.** Equality, J, groupoidal dependent sums/products, and substantial categorical DTT are present, but their complete specialization/closure relationship is not.
- **Metatheory.** No global normalization, confluence, canonicity, consistency, decidability, or semantic-soundness theorem is claimed for the full combined calculus. Lambdapi's local rule checks and the project's diagnostics are implementation evidence, not replacements for those theorems.
- **Higher categories.** The kernel is strict/lax and ω-oriented. It is not a completed formalization of arbitrary weak ω-categories.

These limits are architectural information, not disclaimers added after the fact. Fail-closed elaboration, explicit product manifests, runtime versus proof-time ownership, and the separation of authority from conformance exist to make partial progress reviewable without silently broadening the claim.

10. Conclusion

Functorial Type Theory treats categorical variation as computational structure. Emdash realizes that idea through complementary layers: an outer dependent logical framework and an inner directed dependent calculus of families, totals, sections, dependent homs, and higher action.

Synthetic arrow induction exhibits the mathematical payoff. The canonical Sigma-transport arrow ρ turns reflexive data into a section over all outgoing arrows, and the composition motive normalizes to $q \circ p$. Usable categorical binders exhibit the implementation payoff. Recursive ordinary and displayed variable occurrences compile to explicit internal owners and are checked by one generic TypeScript LF, including object and arrow behavior.

The current system is bounded, but the design question has a concrete answer: readable binders, explicit Core, checked categorical computation, an authoritative Lambdapi kernel, and a client-side reviewer fit into one architecture. The remaining work is to extend its mathematical and transfer coverage without losing that internalized, normalization-first discipline.

References

1. The Univalent Foundations Program. *Homotopy Type Theory: Univalent Foundations of Mathematics*. Institute for Advanced Study, 2013.
2. Kosta Došen. *Cut Elimination in Categories*. Trends in Logic 6. Kluwer Academic Publishers, 1999.
3. The Lambdapi contributors. *Lambdapi User Manual*. lambdapi.readthedocs.io.
4. The emdash contributors. *emdash v3.2 Lambdapi and TypeScript Sources*. Accompanying computational artifact.